TA Session 01

HARVARD EXTENSION SCHOOL

Welcome

- We know that many of you use different options for working!
- The following instructions will work for POSIT Cloud but you can use the Cloud Version OR the downloadable version.

Agenda

Schedule / Syllabus

Assignment Tips

POSIT Cloud

R Basics

Questions about the class?

Schedule

- Classes will be held on Thursdays 5:10 PM – 7:10 PM EST
- TA Sessions will be held on Tuesdays starting at 5:10 PM EST
- Assignments will be due weekly on Friday (expect for Holidays and breaks)

- Midterm: Friday, April 3th at 12 pm EST through Wednesday April 8th at 12:00 pm EST.
  - You must submit your midterm by Wednesday April 8th at 12:00 pm EST.
  - Once you start the exam, you have to complete the exam in 3 hours or by Wednesday April 8th at 12:00 pm EST.
  - Your exam will be an open book and open notes exam.

- Final: Thursday, May 14th at 12 pm EST through Friday May 15th at 12:00 pm EST.
  - You must submit your midterm by Friday May 15th at 12:00 pm EST.
  - Once you start the exam, you have to complete the exam in 3 hours or by Friday May 15th at 12:00 pm EST.
  - Your exam will be an open book and open notes exam.

- Syllabus: https://canvas.harvard.edu/courses/166146/external_tools/90443

Assignments: What to turn in

You must turn in the following for each Assignment:

- Working FULL Code file (rmd)
- AND one of the following:
  - PDF (fully run and producing your output)
  - Html (fully run and producing your output)

Note: Your pdf/html is expected to show all code run and all final assignment values.

To be clear.. You must turn in two items each week:

Item 1: Full code RMD File

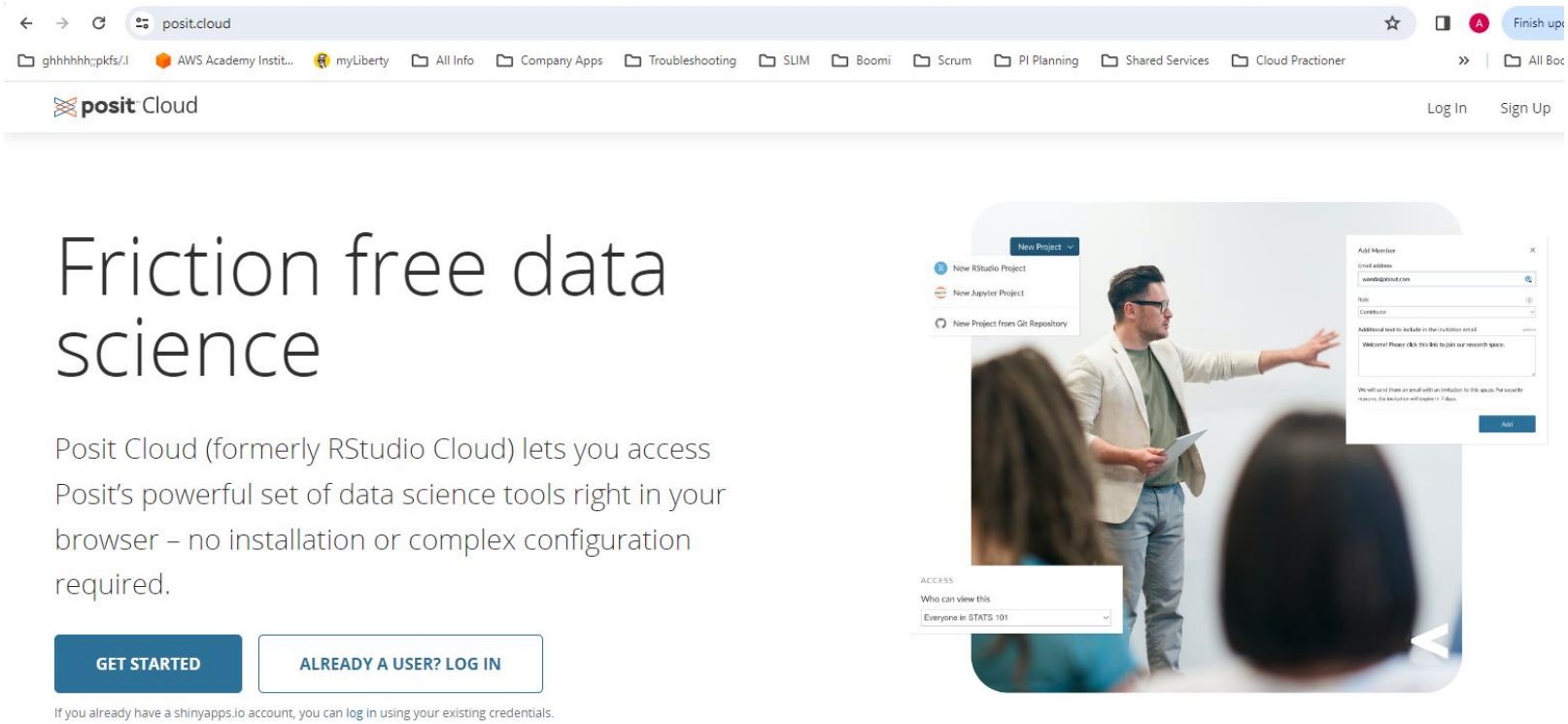
Item 2: A pdf or html file

Posit CLOUD

We use POSIT Cloud because:

- you don't need to have your own powerful computer with administrative rights (can do your homework anywhere, even on a tablet or Chromebook).
- POSIT Cloud is powerful enough (enough memory and disk space).
- You will have the same version of your libraries.
- You will not need individual support since everything is the same (same errors for everyone, and not depending on the version of your system).

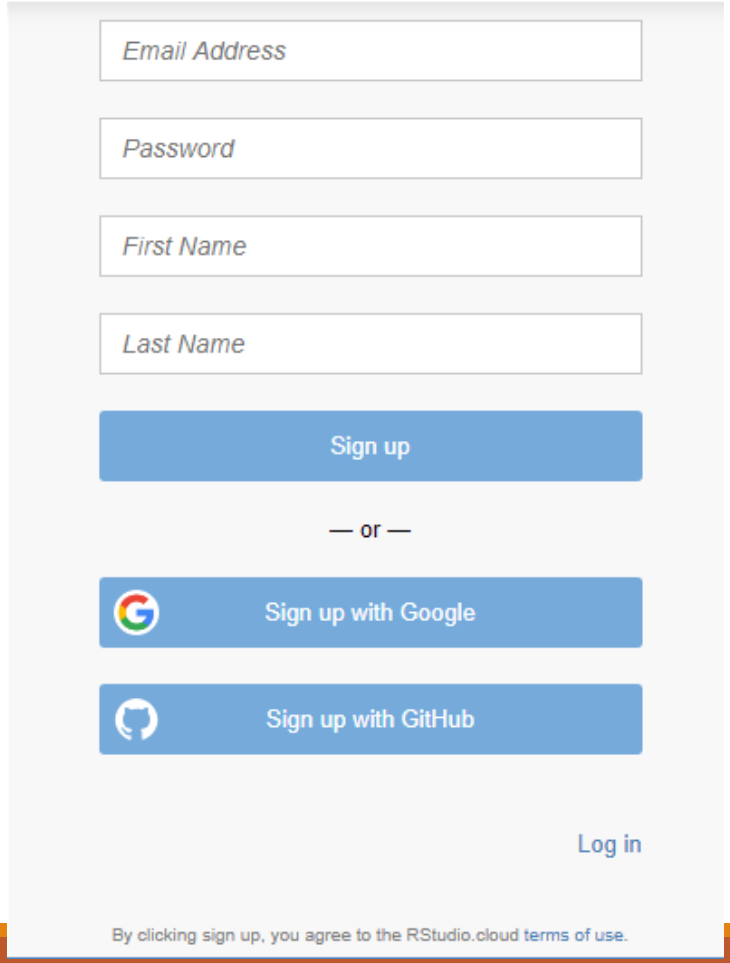
POSIT Cloud


posit.cloud
Finish up
ghhhhhh;;pkfs/.l
AWS Academy Instit...
myLiberty
All Info
Company Apps
Troubleshooting
SLIM
Boomi
Scrum
PI Planning
Shared Services
Cloud Practioner
All Boo
posit Cloud
Log In
Sign Up
Friction free data science
Posit Cloud (formerly RStudio Cloud) lets you access Posit's powerful set of data science tools right in your browser – no installation or complex configuration required.
GET STARTED
ALREADY A USER? LOG IN
If you already have a shinyapps.io account, you can log in using your existing credentials.
New Project
New RStudio Project
New Jupyter Project
New Project from Git Repository
Add Member
Add
ACCESS
Who can view this
Everyone in STATS 101

Create a log-in

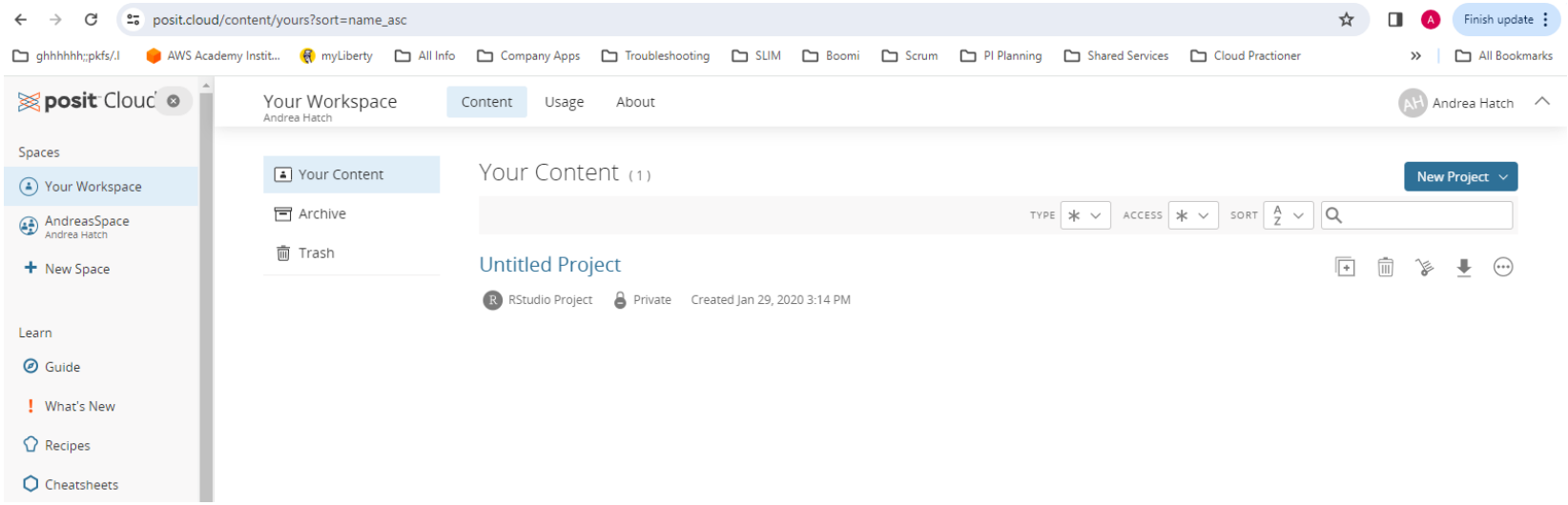
Sign-up or Log in if you already have an account

Note: You can use your google or GitHub account if you have one



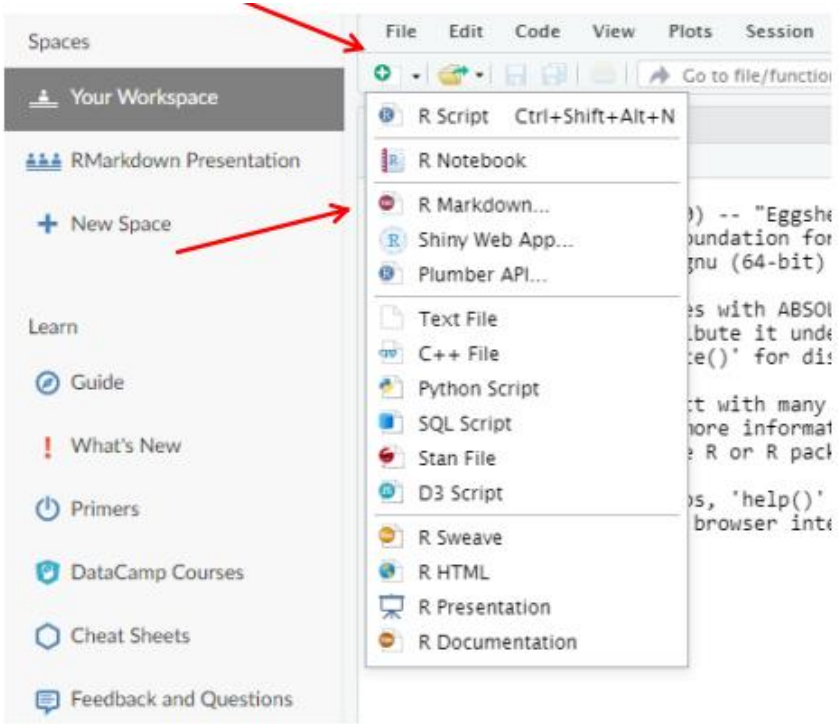
Get Started

To get started select New Project and you are ready to go



Get Started with R markdown

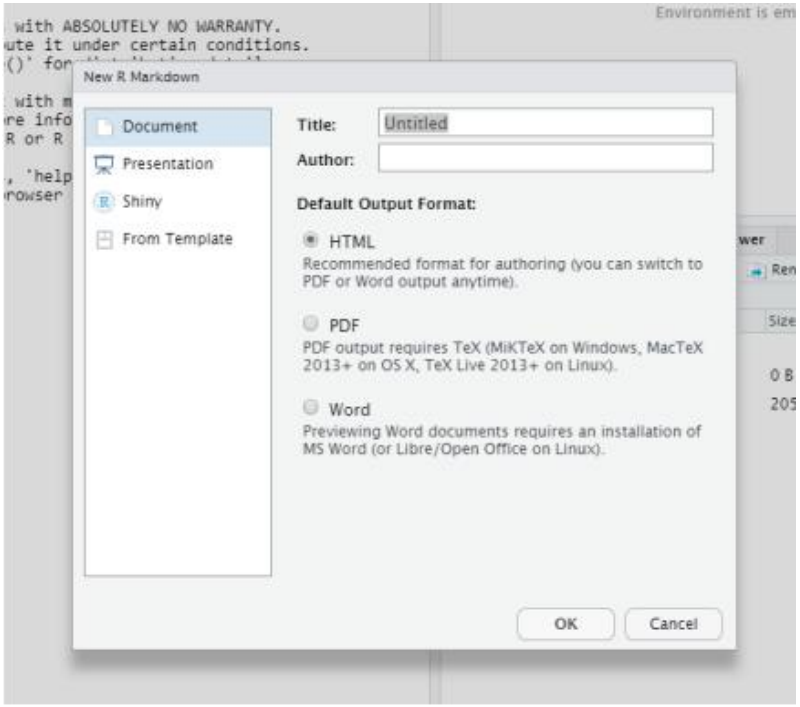
Click the plus button and select R Markdown (this is what is required to be turned in for your homework



Get Started with R markdown

You will need to update packages (click “Yes” when prompted)

Choose title, author name and what is the output



Get Started with R markdown

R Markdown uses so called “literate programming” that mixes plain text and output of executed code

R Markdown also promotes reproducibility because what you don‟t include in the code that won‟t be executed (not like Excel)

1. You can write regular text
2. You can write in LaTeX notation:
   - $$ \beta = (X^TX)^{-1}X^Ty $$ translate to
   - Beta =
3. R code inline (`r 1+1`) 4

$$\beta = (X^T X)^{-1} X^T y$$

4. R code in so-called chunks (starting ```{r} ; ending ```)

- RMarkdown can execute also Python, C++ (RCpp), SQL and others (but we wont use these here)

Get Started with R markdown

See Rmarkdown web page:
https://rmarkdown.rstudio.com/docs/index.html?_gl=1*1sqj32l*_ga*OTI0ODEyOTI2LjE3Mzc5MTIwNjg.*_ga_2C0WZ1JHG0*MTczODI3NDkxOS4yLjEuMTczODI3NTEyNC4wLjAuMA

See the many cheat sheets here:
https://posit.co/resources/cheatsheets/

Textbooks and other resources for learning R

- Paul Gerrard& Radia M. Johnson, Mastering Scientific Computing with R
- Julian Faraway, Linear Models with R
- P. Kuhnert & B. Venables, An Introduction to R: Software for Statistical Modeling & Computing
- Adler, J: R in a Nutshell
- Teetor, P.: R Cookbook
- Check https://cran.r-project.org/ for manuals and other resources/
- If you are an R beginner (no endorsement), we've heard videos from R programming course from JHU/Coursera is good (can be found on youtube)

R as a Calculator

```
> 1 + 1 # Simple Arithmetic
[1] 2

> 2 + 3 * 4 # Operator precedence
[1] 14

> 3 ^ 2 # Exponentiation
 [1] 9
> exp(1) # Basic mathematical functions are available
 [1] 2.718282
> sqrt(10)
 [1] 3.162278
>pi # The constant pi is predefined
 [1] 3.141593
```

R introductions

Results of calculations can be stored in objects using the assignment operators:

- An arrow (<-) formed by a smaller than character and a hyphen without a space!
- The equal character (=).
- X=2
- Y<-2

These objects can then be used in other calculations. To print the object just enter the name of the object. There are some restrictions when giving an object a name:

- Object names cannot contain `strange' symbols like !, +, -, #.
- A dot (.) and an underscore ( ) are allowed, also a name starting with a dot.
- Object names can contain a number but cannot start with a number.
- R is case sensitive, X and x are two different objects, as well as DAT and daT.

R introductions, *cont'd*

- To list the objects that you have in your current R session use the function ls or the function objects.
  - `> ls()`
  - `[1] "x" "y" "Y" "z"`
- If you assign a value to an object that already exists then the contents of the object will be overwritten with the new value (without a warning!).
  - `X<-2`
  - `X<-3`
- Use the function rm to remove one or more objects from your session.
  - `> rm(x)` #trick to remove everything: rm(list=ls())

R introductions, *cont'd*

The collection of objects that you currently have is called the workspace and it is not automatically saved by R. Always save your workspace !

R gets confused if you use a path in your code like

```
c:\mydocuments\myfile.txt
```

- This is because R sees "\" as an escape character. Instead, use

```
c:\\my documents\\myfile.txt
```

  - or

```
c:/mydocuments/myfile.txt
```

- Once R is installed, there is a comprehensive built-in help system. At the program's command prompt you can use any of the following:

```
help.start()   # general help
help(ls)      # help about function ls
?ls           # same thing
apropos("ls") # list all function containing string ls
example(ls)   # show an example of function ls
```

R introductions, *cont'd*

- R comes with a number of sample datasets that you can experiment with.
- Type `data()` to see the available datasets. The results will depend on which packages you have loaded.
  - `> data()`

Data sets in package 'datasets':	
AirPassengers	Monthly Airline Passenger Numbers 1949-1960
BJsales	Sales Data with Leading Indicator
BJsales.lead (BJsales)	Sales Data with Leading Indicator
BOD	Biochemical Oxygen Demand
CO2	Carbon Dioxide Uptake in Grass Plants
ChickWeight	Weight versus age of chicks on different diets

- Type **`help(`***`datasetname`***`)`** for details on a sample dataset.
  - `> help(AirPassengers)`

R introductions, *cont'd*

R is an open source. You can write new functions and package those functions in a so called `R package' (or `R library') or you can use R packages written by others.

There is a lively R user community and many R packages have been written and made available on CRAN for other users.

```
library()  # to see what R packages you have

install.packages("MASS") # install MASS package

install.packages("faraway") # install faraway package

library("faraway") # to load a library
```

R Vectors

A series of numbers

Created with

- c() to concatenate elements or sub-vectors
- rep() to repeat elements or patterns
- seq() or m:n to generate sequences
- Examples:
  - `x <- c(1, 2, 3)`
  - `[1] 1 2 3`

  - `y <- seq(1:3)`
  - `[1] 1 2 3`

  - `z <- rep(3, 14)`
  - `[1] 3 3 3 3 3 3 3 3 3 3 3 3 3 3`

Defining Vectors

```
> rep(1,10) # repeats the number 1, 10 times
  [1] 1 1 1 1 1 1 1 1 1 1

 > seq(2,6) # sequence of integers between 2 and 6
  [1] 2 3 4 5 6 # equivalent to 2:6

> seq(4,20,by=4) # Every 4th integer between 4 and 20
  [1] 4 8 12 16 20

> x <- c(2,0,0,4) # Creates vector with elements 2,0,0,4

> y <- c(1,9,9,9)

> x + y # Sums elements of two vectors
  [1] 3 9 9 13

> x * 4 # Multiplies elements
  [1] 8 0 0 16

> sqrt(x) # Function applies to each element
  [1] 1.41 0.00 0.00 2.00 # Returns vector
```

Operations on vectors: example

For example, if we multiply a vector by 2, all the elements of the vector will be multiplied by 2

- `> x <- c(1, 3, 5, 10) > x * 2`
- `[1] 2 6 10 20`
- `> x <- c(1, 3, 5, 10) ; y <- c(13, 15, 17, 22) ; x + y`
  - `[1] 14 18 22 32`
- If the vectors are of different lengths, the shorter vector will be extended to match the length of the longer vector by recycling its elements starting from the first element. However, you will also get a warning message from R in case you did not intend to add vectors of differing length, as follows:
  - `z <- c(1,3, 4, 6, 10) ; x + z #1 was recycled to complete the operation.`
    - `[1] 2 6 9 16 11 Warning message: In x + z : longer object length is not a multiple of shorter object length`
  - the standard operators also have %%, which indicates x mod y, and %/%, which indicates integer division as follows:
    - `> x %% 2`
      - `[1] 1 1 1 0`
    - `> x %/% 5`
      - `[1] 0 0 1 2`

Lists

A list is a generalization of a vector and represents a collection of data objects. A list allows you to gather a variety of (possibly unrelated) objects under one name. Here are some examples:

```
hd<- list(name="John Smith", id=1111, grade="B+", age=35)
```

```
> hd
$name
[1] "John Smith"
$id
[1] 1111
$grade
[1] "B+"
$age
[1] 35
```

Four levels: name, id, grade, and age

- Identify elements of a list using the [[]] convention.

```
hd[[2]] # 2nd component of the list
  [1] 1111
```

Factors

Categorical data can also be stored as Factors. These are specialized vectors that contain predefined values referred to as Levels.

For example, say you have data for "placebo" and "treatment" for four patients, you could store this information as factors instead of a character vector by using the following code:

```
drug_response <- c("placebo", "treatment", "placebo", "treatment")

levels(drug_response)
```

- `NULL  # R did not store drug_response as a factor.`

```
drug_response <- factor(drug_response)    #factor function or
as.factor function will store it as a factor

levels(drug_response)
```

- `[1] "placebo"   "treatment"`

To check the integers used for each level, you can use the as.integer() function as follows:

```
> as.integer(drug_response) [1] 1 2 1 2
```

Matrices

All columns in a matrix must have the same mode(numeric, character, etc.) and the same length. The general format is

```
mymatrix <- matrix(vector, nrow=r, ncol=c, byrow=FALSE,
dimnames=list(char_vector_rownames, char_vector_colnames))
```

- `byrow=TRUE` indicates that the matrix should be filled by rows.
- `byrow=FALSE` indicates that the matrix should be filled by columns (the default).
- `dimnames` provides optional labels for the columns and rows.
- Example: what are the differences between two matrices?
  - `y<-matrix(1:20,byrow=F, nrow=5,ncol=4)`
  - `y<-matrix(1:20,byrow=T, nrow=5,ncol=4)`

Multi-dimensional arrays

Arrays are similar to matrices but can have more than two dimensions. See **help(array)** for details.

```
list_example = list((1:2), (1:5))
[[1]]
[1] 1 2 3


[[2]]
[1] 1 2 3 4 5
```

Data Frames

The most common way to store data in R is through data frames and it makes data analysis much easier, especially when dealing with categorical data.

Data frames are similar to matrices, except that each column can store different types of data.

You can construct data frames using the `data.frame()` function or convert an R object into a data frame using the `as.data.frame()` function as follows:

```
students <- c("John", "Mary", "Ethan", "Dora")
score<- c(76, 82, 84, 67)
grade <- c("B", "A", "A", "C")
class <- data.frame(students, score, grade)
```

```
> class
  students score grade
1     John    76     B
2     Mary    82     A
3    Ethan    84     A
4     Dora    67     C
```

Useful functions

```
length(object) # number of elements or components
str(object)    # structure of an object
class(object)  # class or type of an object
names(object)  # names
c(object1,object2,...)  # combine objects into a
  vector
cbind(object,1 object2, ...) # combine objects as
  columns
rbind(object,1 object,2 ...) # combine objects as
  rows
ls()       # list current objects
rm(object) # delete an object
newobject <- edit(object) # edit copy and save a
newobject
fix(object)
```

Loading data into r

You can see where R will read and save files, by default, using the `getwd()` function. Then, you can change it using the `setwd()` function

```
getwd()
```

- `[1] "/cloud/project“`

- Use `read.table(), read.delim(), read.csv()` functions to load the delimited data into R
  - `myData.df <- read.table("myData.txt", header=TRUE, sep="\t")`
  - `read.delim("myData.txt", header=TRUE)`
  - `myData2.df <-read.csv("myData.csv", header=FALSE)`

- To load excel files first install `gdata` package and use read.xls function
  - `install.packages("gdata")`
  - `myData.df <- read.xls("myData.xlsx", sheet=1) #also uploads .xls files and returns a data frame`

Saving / exporting files

To save an object, preferably a matrix or data frame, you can write a .txt or .csv file or a file using another delimiter using the `write.table()` or `write.csv()` functions.

You can choose to include `row.names` and `col.names` by setting these arguments to TRUE.

The output file will be saved to your current directory.

- `write.table(myData.df, file="savedata_file.txt", quote = FALSE, sep= "\t", row.names=TRUE, col.names=NA, append=FALSE)`
- `write.csv(myData.df, file = "savedata_file.csv") #same output as above`

Missing Data

In **R**, missing values are represented by the symbol **NA** (not available) .

- `y <- c(1,2,3,NA)`
- `is.na(y) # returns a vector (F F F T)`
- `na.omit(y) # delete missing values`

Arithmetic functions on missing values yield missing values.

- `mean(y)`
- `[1] NA`
- `mean(y, na.rm=TRUE)  # na.rm removes the missing values`
- `[1] 2`

Flow control

- **for**
  - `for (var in seq) {expr}`
- **if-else**
  - `if (cond) {expr}`
    `if (cond) {expr1} else {expr2}`
- **ifelse**
  - `ifelse(test,yes,no)`
- **while**
  - `while (cond) {expr}`
- **switch**
  - `switch(expr, ...)`

The for() loop

The `for(i in vector){commands}` statement allows you to repeat the code written in brackets {} for each element (i) in your vector in parenthesis.

- `for(i in 1:5){x[i] <- x[i+2] + x[i+1]}`
- Example: Write for loop function that calculates the cumulative total for each row or element

if() and else if () statements

The `if(condition){commands}` statement allows you to evaluate a condition and if it returns TRUE, the code in brackets will be executed.

You can add an else {commands} statement to your if() statement if you would like to execute a block of code if your condition returns FALSE:

```
x <- 4 > # we indent our code to make it more legible
if(x < 10) { x <-x+4; print(x) }
[1] 8
```

If you have several conditions to test before running an else {} statement, you can use an else if(condition){commands} statement as follows:

```
x <- 1
if(x == 2) { x <- x+4; print("X is equal to 2, so I added 4 to it.") } else if (x > 2) { print("X is greater than 2, so I did nothing to it.") } else { x <- x -4 ;print("X is not greater than or equal to 2, so I subtracted 4 from it.") }
[1] "X is not greater than or equal to 2, so I subtracted 4 from it."
```

Sorting

- To sort a dataframe in R, use the order( ) function. By default, sorting is ASCENDING. Prepend the sorting variable by a minus sign to indicate DESCENDING order. Here are some examples.
- ```
  data(mtcars)
  # sort by mpg
  newdata = mtcars[order(mtcars$mpg),]
  # sort by mpg and cyl
  newdata <- mtcars[order(mtcars$mpg, mtcars$cyl),]
  #sort by mpg (ascending) and cyl (descending)
  newdata <- mtcars[order(mtcars$mpg, -mtcars$cyl),]
  ```
  ```

Merging

- To merge two dataframes (datasets) horizontally, use the merge function. In most cases, you join two dataframes by one or more common key variables (i.e., an inner join).
- ```
  # merge two dataframes by ID
  total <- merge(dataframeA,dataframeB,by="ID")
  ```
- ```
  # merge two dataframes by ID and Country
  total <- merge(dataframeA,dataframeB,by=c("ID","Country"))
  ```

Merging – adding rows

- To join two dataframes (datasets) vertically, use the rbind function. The two dataframes must have the same variables, but they do not have to be in the same order.
- `total <- rbind(dataframeA, dataframeB)`
  - If dataframeA has variables that dataframeB does not, then either:
  - Delete the extra variables in dataframeA or
  - Create the additional variables in dataframeB and set them to NA (missing)
  - before joining them with rbind.

Writing R Functions

Functions are created using the `function()` directive and are stored as R objects just like anything else. In particular, they are R objects of class "function".

```
f <- function() { ## Do something interesting }
```

Functions in R are "first class objects", which means that they can be treated much like any other R object. Importantly, Functions can be passed as arguments to other functions

Functions can be nested, so that you can define a function inside of another function

The return value of a function is the last expression in the function body to be evaluated.

Questions?

- How does everyone feel about R?

- Questions about the class?